

Segurança em Aplicações Web: Vulnerabilidades Comuns e Defesas

Ana da Silva, Fabricio Fernandes, Matheus Gussiardi, Matheus Izidio, Ubiratã Costa

¹ Centro de Matemática, Computação e Cognição (CMCC)
Universidade Federal do ABC (UFABC)
Av. dos Estados, 5001, Bangu, Santo André/SP, Brazil

Resumo. *O trabalho analisa vulnerabilidades do OWASP Top 10 sob o princípio da defesa em profundidade, unificando ferramentas SAST, DAST, IAST e RASP. A revisão bibliográfica apoiada por IA revelou lacunas em shadow APIs e na interpretabilidade de IA para detecção. Conclui-se pela necessidade de automação via DevSecOps arquiteturas Zero Trust.*

Palavras-chave: *Segurança Web, Defesa em Profundidade, DevSecOps, Zero Trust.*

Abstract. *This study analyzes OWASP Top 10 vulnerabilities through defense-in-depth, integrating SAST, DAST, IAST, and RASP. Literature review with GenAI support identified gaps in shadow API security and AI interpretability. Results point to DevSecOps automation and Zero Trust architectures.*

Keywords: *Web Security, Defense-in-Depth, DevSecOps, Zero Trust.*

1. Introdução

A transformação digital ampliou o uso de aplicações web e mobile e, com isso, a exposição de dados e serviços críticos.

Aplicações concentram dados sensíveis e regras de negócio; por isso, vulnerabilidades nessa camada se tornaram alvo frequente.

Falhas na validação e sanitização de entradas permitem ataques graves, enquanto o foco exclusivo em defesas de rede já não é suficiente diante de ameaças que exploram diretamente a aplicação.

Neste cenário, este trabalho apresenta fundamentos de segurança em aplicações web, examina vulnerabilidades críticas — como injeção de código, XSS e falhas de controle de acesso —, sintetiza mecanismos de defesa e práticas de mitigação e discute tendências para sistemas de larga escala, integração contínua e uso de IA na detecção de falhas.

2. Revisão Bibliográfica

2.1. Fundamentos da Arquitetura Web

A Arquitetura Orientada a Serviços (SOA) organiza sistemas distribuídos na interação entre cliente, provedor e registro de serviços, normalmente no ciclo *publish-find-bind* [Mello et al. 2006]. Nesse modelo, SOAP, WSDL e UDDI sustentam troca de mensagens, descrição de interfaces e descoberta de serviços.

Com a evolução das aplicações web, a adoção de microsserviços ampliou a granularidade e o uso de protocolos leves, como REST [Pahl and Jamshidi 2016]. Essa transição, detalhada na Seção 4.3, aumentou a complexidade de segurança em comunicação entre serviços e exposição de APIs.

Além de ganhos de escalabilidade e manutenção, essa mudança arquitetural deslocou parte dos riscos para interfaces distribuídas e para a governança de dependências entre serviços. Em ambientes modernos, autenticação entre componentes, rastreabilidade de chamadas e consistência de políticas de acesso tornaram-se requisitos operacionais para reduzir superfície de ataque.

2.2. Conceitos de Segurança

A segurança da informação em aplicações web é sustentada por cinco requisitos: confidencialidade, integridade, disponibilidade, autenticidade e não-repúdio [Mello et al. 2006]. Esses princípios se refletem em criptografia, autenticação, autorização e mecanismos de continuidade de serviço.

Como aplicações web processam entradas externas e dados sensíveis, elas permanecem expostas a múltiplos vetores de ataque [Yadav et al. 2018]. Para padronizar análise e priorização, destacam-se taxonomias como o OWASP *Top 10* [OWASP Foundation 2021] e classificações sistemáticas de exploração de falhas [Shahriar and Zulkernine 2011].

Essas taxonomias orientam avaliação de risco, requisitos de segurança e priorização de correções. Assim, a revisão bibliográfica descreve vulnerabilidades recorrentes e estrutura critérios para comparar estratégias de defesa.

2.3. Vulnerabilidades e Mecanismos de Defesa

As vulnerabilidades em aplicações web são frequentemente agrupadas por vetor de ataque. Entre as mais recorrentes estão injeções (como SQLi), *Cross-Site Scripting* (XSS), falhas de controle de acesso, *Cross-Site Request Forgery* (CSRF), más configurações e problemas em componentes de terceiros [Halfond et al. 2006, Gupta and Gupta 2015, Alwan and Younis 2021].

As defesas atuam em camadas. No nível preventivo, destacam-se validação/sanitização de entradas, parametrização de consultas e *tokens* anti-CSRF [Shar and Tan 2016]. No ciclo de desenvolvimento, SAST, DAST e IAST identificam falhas em diferentes fases. Em produção, WAF e RASP ampliam a proteção em tempo de execução. A Seção 4.2 aprofunda essas técnicas.

Nenhuma técnica isolada cobre todo o ciclo de vida da aplicação. Medidas preventivas reduzem falhas, testes especializados antecipam detecção e controles em *runtime* mitigam exploração residual. Em conjunto, essas práticas sustentam uma postura contínua de segurança.

3. Metodologia

REVISAR TODO MUNDO JUNTO

O levantamento bibliográfico foi realizado via *Google Scholar* com base na bibliografia sugerida, priorizando artigos de periódicos e anais de conferências da área. Fer-

ramentas de IA Generativa **citar ferramentas usadas** foram empregadas como suporte na síntese inicial de conceitos e estruturação da visão geral do tema, para auxiliar a redação.

A redação foi dividida entre os autores, que trabalharam individualmente no desenvolvimento dos tópicos e posteriormente discutiram o texto final, sugerindo melhorias.

Toda a fundamentação técnica e o desenvolvimento do projeto foram validados e conduzidos pelos autores, garantindo o rigor científico e a integridade dos dados.

4. Discussão

Entre os principais conceitos utilizados para estruturar a análise de vulnerabilidades e métodos de mitigação estão a taxonomia de vulnerabilidades, os mecanismos de segurança e as arquiteturas de segurança. Esses três elementos desempenham papéis complementares e inter-relacionados nos processos de identificação, prevenção e mitigação de falhas em aplicações web.

4.1. Taxonomia de Vulnerabilidades

A taxonomia de vulnerabilidades permite organizar falhas de segurança em categorias estruturadas, facilitando sua análise e mitigação. Nesse contexto, diferentes *frameworks* e organizações especializadas desenvolveram modelos de classificação amplamente adotados pela comunidade de segurança da informação. Entre eles, destaca-se o trabalho da *OWASP Foundation*, referência para identificação e priorização de vulnerabilidades em aplicações web.

Mecanismos de segurança correspondem a técnicas e controles para prevenir, detectar e mitigar falhas, como validação de entrada, autenticação multifator, criptografia, controle de acesso e monitoramento de *logs*. Arquiteturas de segurança, por sua vez, descrevem como esses mecanismos são integrados no sistema, estabelecendo camadas de proteção. Estratégias como *defense in depth* e isolamento de componentes em microsserviços exemplificam esse arranjo em aplicações modernas.

4.1.1. Framework OWASP Top 10: evolução 2017 → 2021

Entre 2017 e 2021, a classificação do OWASP Top 10 foi reordenada. *Broken Access Control* passou ao primeiro lugar, evidenciando a necessidade de controles de autorização mais rigorosos ao longo de todo o ciclo de desenvolvimento. Falhas criptográficas subiram para a segunda posição, enquanto injeção caiu para a terceira (incidência máxima de 19% e média de 3,37%). Além disso, a categoria *Insecure Design* reforçou a importância de decisões arquiteturais seguras desde as fases iniciais, evitando dependência de validações apenas no *frontend* [OWASP Foundation 2021].

4.1.2. Vulnerabilidades de Injeção (*SQLi*, *Command*, *LDAP*)

Vulnerabilidades de injeção ocorrem quando entradas do usuário são interpretadas como comandos por subsistemas, como banco de dados, sistema operacional ou diretórios LDAP. A mitigação requer validação e sanitização rigorosas, uso de abstrações seguras (como ORM), princípio do menor privilégio e monitoramento contínuo de *logs*. Em

SQL injection (SQLi), entradas manipuladas alteram consultas (por exemplo, com tautologias como `OR 1=1`), permitindo leitura ou alteração indevida de dados [Appelt 2014]. Já *command injection* pode levar à execução arbitrária no sistema operacional, e *LDAP injection* permite alterar a lógica de consultas a diretórios, resultando em acesso não autorizado.

4.1.3. Cross-Site Scripting (XSS): Reflected, Stored, DOM-based

Cross-Site Scripting (XSS) permite a injeção de *scripts* maliciosos em páginas visualizadas por outros usuários quando dados de entrada não são devidamente tratados [Sarmah et al. 2018]. No *Reflected XSS*, o conteúdo malicioso é retornado imediatamente na resposta; no *Stored XSS*, ele é persistido no servidor e executado para múltiplos usuários; no *DOM-based XSS*, a exploração ocorre no navegador por manipulação insegura do DOM em *JavaScript*. Os impactos incluem roubo de sessão, redirecionamentos maliciosos, execução de ações em nome da vítima e disseminação de *malware*.

4.1.4. Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) explora a confiança da aplicação no navegador autenticado da vítima. O atacante induz o usuário a disparar uma requisição maliciosa para um sistema onde já possui sessão ativa; como *cookies* de autenticação são enviados automaticamente, o servidor tende a interpretar a ação como legítima [Lin et al. 2009].

4.1.5. Broken Access Control e IDOR

Broken Access Control ocorre quando regras de autorização são aplicadas de forma incompleta, permitindo acesso ou ações além do perfil do usuário [Kusnana et al. 2025]. Por exemplo, um *endpoint* administrativo como `https://site.com/admin/financas` torna-se acessível sem validação adequada de permissões. O *Insecure Direct Object Reference (IDOR)* é um caso recorrente: ao expor identificadores em URLs ou parâmetros sem checagem de autorização, a aplicação permite manipulação de IDs e acesso indevido a recursos de terceiros, como em `https://site.com/pedido?id=1025`.

4.1.6. Falhas Criptográficas

Falhas criptográficas decorrem do uso inadequado de mecanismos de proteção de dados sensíveis. Entre as causas mais comuns estão algoritmos obsoletos (como SHA-1, DES e 3DES), gestão incorreta de chaves, geração previsível de números aleatórios e reutilização de vetores de inicialização, fatores que podem comprometer completamente a confidencialidade [Blessing et al. 2021]. Também são recorrentes o armazenamento inseguro de senhas (*plain text* ou hash inadequado) e erros de transmissão, como ausência de HTTPS ou configuração incorreta de certificados, facilitando ataques *man-in-the-middle* (MitM).

4.1.7. *Misconfiguration e Supply Chain*

Más configurações (*misconfiguration*) surgem quando sistemas são implantados com parâmetros inseguros, por ausência de padronização ou erro operacional. Exemplos incluem *buckets* públicos, portas administrativas expostas, bancos sem autenticação adequada e credenciais padrão. Em ambientes *cloud*, esse problema permanece entre as principais causas de violação de dados [Cloud Security Alliance 2024]. Ataques de *supply chain*, por sua vez, exploram relações de confiança em bibliotecas, *frameworks* e pipelines de CI/CD: ao comprometer um componente amplamente usado, o atacante pode propagar código malicioso em larga escala, como no caso *event-stream* (NPM, 2018), com impacto sobre confidencialidade, integridade e disponibilidade.

4.2. Mecanismos de Defesa

Os mecanismos de defesa complementam a taxonomia de vulnerabilidades ao reunir controles de prevenção, detecção e mitigação de ataques. Esta seção apresenta defesas por categoria, ferramentas de análise e soluções em *runtime*.

4.2.1. Defesas por categoria

Cada categoria de vulnerabilidade possui mecanismos de defesa com efetividade e limitações distintas. A Tabela 1 relaciona técnicas, ataques mitigados e restrições com base em [Halfond et al. 2006], [Gupta and Gupta 2015], [Barth et al. 2008] e [OWASP Foundation 2021].

4.2.2. Ferramentas de análise: SAST, DAST, IAST

As ferramentas de análise identificam vulnerabilidades ao longo do ciclo de vida da aplicação. O SAST (*Static Application Security Testing*) analisa código ou *byte-code* sem execução e é adequado para CI/CD [Alazmi and De Leon 2022]. O DAST (*Dynamic Application Security Testing*) testa a aplicação em execução, enquanto o IAST (*Interactive Application Security Testing*) combina instrumentação dinâmica com evidências do código para reduzir falsos positivos [Cotroneo 2025]. Evidências recentes indicam que a combinação das três abordagens amplia cobertura e efetividade [De Jesús Domínguez-García et al. 2023].

4.2.3. Defesas em *runtime*: WAF, WAAP, RASP

Defesas em *runtime* atuam em produção. O WAF (*Web Application Firewall*) filtra tráfego HTTP por regras e assinaturas, com limitações típicas de abordagens *black-box* [Doupé et al. 2010]. O WAAP amplia a proteção para APIs, e o RASP (*Runtime Application Self-Protection*) atua dentro da aplicação, usando contexto de execução para elevar a precisão e mitigar ataques *zero-day* [Sureda Riera et al. 2025].

Tabela 1. Técnicas de defesa por categoria de ataque e limitações

Técnica	Ataque	Limitações
Prepared Statements/ ORM	Injeção (SQLi; LDAP)	Requer refatoração; não cobre lógica de negócio vulnerável
Validação positiva de entrada	Injeção (SQLi; LDAP; Command)	Listas de permissão podem ficar desatualizadas; bypass por encoding
Content Security Policy (CSP)	XSS (Reflected; Stored; DOM)	Configuração complexa; incompatibilidade com bibliotecas legadas
Encoding contextual/ Sanitização	XSS	Dependente do contexto (HTML; JS; URL); mXSS em innerHTML [Heiderich et al. 2013]
HTTPOnly/ Secure cookies	Roubo de sessão via XSS	Não impede CSRF; XSS ainda pode executar ações em nome do usuário
CSRF Token/ SameSite/ Double Submit	CSRF	Token em APIs stateless; SameSite=Lax não cobre GET maliciosos
RBAC/ ABAC/ Validação backend	Broken Access Control; IDOR	Modelagem de permissões complexa; validação deve ser consistente
TLS 1.3/ bcrypt-Argon2/ Criptografia em repouso	Falhas criptográficas	Configuração incorreta; algoritmos fracos em sistemas legados
IaC scanning/ SBOM/ Lockfiles	Misconfiguration; Sup- ply Chain	Cobertura parcial; dependências transitivas

Fonte: [Halfond et al. 2006]; [Heiderich et al. 2013]; [Gupta and Gupta 2015]; [Barth et al. 2008]; [OWASP Foundation 2021].

Tabela 2. Comparativo SAST, DAST, IAST

Critério	SAST	DAST	IAST
Momento da análise	Pré-execução (código)	Runtime (aplicação em execução)	Durante testes funcionais
Cobertura de fluxo	Parcial (sem contexto runtime)	Completa (tráfego real)	Alta (instrumentação)
Falsos posi- tivos	Maior	Intermediário	Menor [Cotroneo 2025]
Integração CI/CD	Nativa	Requer ambiente im- plantado (deployed)	Requer agente no processo

Fonte: [Alazmi and De Leon 2022]; [Sacramento et al. 2024]; [De Jesús Domínguez-García et al. 2023]; [Cotroneo 2025].

4.2.4. Comparativo WAF × RASP

[Sureda Riera et al. 2025] compararam WAF e RASP em diferentes categorias de ataque. Os resultados indicam maior precisão do RASP, em grande parte pelo conhecimento contextual da aplicação. A Tabela 3 sintetiza esse comparativo.

Tabela 3. Comparativo empírico WAF × RASP

Categoria de ataque	WAF (efetividade)	RASP (efetividade)	Observação
SQL Injection	Média (assinaturas)	Alta (contexto de query)	RASP identifica ponto de injeção
XSS	Média (padrões HTML/JS)	Alta (contexto de saída)	RASP conhece fluxo de dados
Command Injection	Variável	Alta	RASP monitora chamadas de sistema
Path Traversal	Média	Alta	RASP intercepta acesso a arquivos
Zero-day/variações	Baixa	Maior resiliência	RASP baseado em comportamento

Fonte: [Alazmi and De Leon 2022]; [Sacramento et al. 2024]; [De Jesús Domínguez-García et al. 2023]; [Cotroneo 2025].

Os autores recomendam o uso complementar: WAF na borda para filtragem e mitigação de DDoS, e RASP para proteção contextual contra explorações que ultrapassem a camada perimetral.

4.3. Segurança de Arquiteturas Modernas

A evolução dos sistemas web para arquiteturas modernas, como microsserviços e aplicações baseadas em nuvem, transformou significativamente a superfície de ataque. Com a fragmentação de aplicações monolíticas em serviços web distribuídos, a comunicação entre processos e a exposição de APIs tornaram-se centrais para o funcionamento e, conseqüentemente, para a segurança dessas aplicações.

4.3.1. APIs REST — OWASP API Top 10, BOLA, *rate limiting*

As APIs REST são predominantes em aplicações modernas e concentram riscos relevantes. Segundo o OWASP API Top 10 (2023), a vulnerabilidade de maior destaque é *Broken Object Level Authorization* (BOLA), que ocorre quando o backend expõe identificadores de objetos sem validar se o usuário autenticado possui permissão para acessá-los. Esse cenário permite leitura, alteração ou exclusão indevida de dados de terceiros.

Outro ponto crítico é a ausência de *rate limiting*, associada ao *Unrestricted Resource Consumption* (Top 4 em 2023). Sem limites de volume e frequência, requisições podem esgotar recursos da aplicação e de serviços dependentes, degradando disponibilidade e ampliando a superfície de abuso.

4.3.2. APIs GraphQL — introspecção, *query depth*, DoS

O GraphQL, embora amplamente adotado, introduz riscos específicos de segurança. Como o cliente define a complexidade da requisição em um único *endpoint* (geralmente `/graphql`), consultas sintaticamente válidas podem impor alto custo computacional e causar DoS. Um vetor recorrente é a *query depth*, isto é, o nível de aninhamento dos campos. A mitigação inclui validação estrutural da consulta e análise da *Abstract Syntax Tree* (AST) antes da execução.

A introspecção, recurso nativo do GraphQL, permite descobrir toda a estrutura da API. Em produção, sua exposição facilita ataques mais precisos, incluindo BOLA e consultas profundas para exaustão de recursos. Por isso, recomenda-se desativar a introspecção em ambientes produtivos.

4.3.3. Microserviços — mTLS, API Gateway, gestão de segredos

O artigo "Zero Trust Security Architecture in Microservices via mTLS" (IJARCSE, 2025) propõe eliminar a confiança implícita em redes distribuídas. Nesse modelo, o mTLS (*Mutual Transport Layer Security*) é o pilar da arquitetura Zero Trust, garantindo criptografia e autenticação mútua entre serviços.

Sidecars (como Envoy) validam certificados a cada conexão, enquanto o API Gateway centraliza o controle de acesso externo e valida tokens JWT com o provedor de identidade (por exemplo, Keycloak). Após validação, o tráfego segue para os microserviços por canais mTLS. A gestão de segredos é reforçada por rotação frequente de certificados X.509 (como no Citadel/Istio) e uso de tokens de curta duração, reduzindo a janela de exploração em caso de comprometimento.

4.3.4. Cloud-native — IaC scanning, Kubernetes, S3

Aplicações *cloud-native* dependem de infraestrutura dinâmica e altamente configurável, o que amplia riscos de *misconfiguration*. Um cenário recorrente é a exposição pública indevida de recursos em provedores de nuvem, como *buckets* S3, possibilitando acesso não autorizado a dados sensíveis. A mitigação exige *hardening* contínuo e revisão sistemática de permissões (ACLs e políticas de acesso).

Esse ecossistema também depende de IaC e *pipelines* de CI/CD, alinhando-se ao risco de integridade de software destacado pela OWASP (A08:2021). Sem controles robustos, *pipelines* podem introduzir código malicioso ou acessos indevidos. Defesas recomendadas incluem governança de acesso ao CI/CD, verificação de dependências por *Software Composition Analysis* e *scanning* de IaC para prevenir falhas de provisionamento.

5. Tendências e Pesquisas em Andamento

As tecnologias de desenvolvimento e as ameaças a aplicações web evoluem em ritmo semelhante, impulsionando pesquisas em novas abordagens de proteção. A mitigação de

ataques complexos exige a incorporação contínua da segurança, do design à operação. A seguir, apresentam-se tendências e frentes de pesquisa atuais.

5.1. Inteligência Artificial para Detecção de Ameaças

A inteligência artificial, especialmente o aprendizado de máquina, tem ampliado a capacidade de detecção de vulnerabilidades e de classificação de anomalias. Modelos como CNN, LSTM e GNN, combinados com técnicas como N-Grams e *support vector machines* (SVM), aprimoram a identificação de padrões de injeção, incluindo XSS e SQL. Também se destacam abordagens comportamentais voltadas a ataques *zero-day* e estudos de robustez adversarial. Como desafio central, permanece a interpretabilidade dos modelos em IDS, de modo que decisões de bloqueio sejam auditáveis por equipes de segurança.

5.2. Arquiteturas de Confiança Zero e Gestão Avançada de Identidades

Com a consolidação de microsserviços e APIs distribuídas, arquiteturas de *Zero Trust* tornam-se essenciais. Nesse modelo, nenhuma requisição é considerada confiável por padrão, exigindo verificação contínua de identidade, dispositivo e contexto, inclusive em tráfego interno. Como aplicações atravessam múltiplos domínios, faz-se necessária a federação de identidades e o estabelecimento dinâmico de confiança. Padrões como SAML 2.0 e XACML permanecem relevantes para SSO e autorização baseada em atributos, com uso de identificadores opacos para reduzir rastreabilidade e reforçar a privacidade.

5.3. DevSecOps - Integração de segurança no CI/CD

A adoção de *DevSecOps* busca inserir segurança de forma nativa no ciclo de desenvolvimento, com verificações automatizadas de integridade e vulnerabilidades ao longo do CI/CD. A integração de SAST, DAST e IAST antecipa a identificação de falhas de injeção e erros lógicos antes da produção, reduzindo o custo de correção. Essa estratégia responde à necessidade de entregas ágeis sem perda de controle e reforça o princípio *shift-left*, em consonância com o destaque de "design inseguro" na OWASP *Top 10* (2021).

5.4. Segurança em Cadeias de Suprimento e Componentes de Terceiros

O uso intensivo de bibliotecas de terceiros, repositórios abertos e CDNs amplia os riscos na cadeia de suprimento de software. Pesquisas atuais priorizam o monitoramento contínuo de dependências, a validação de procedência e a verificação de integridade por assinaturas digitais e ferramentas como OWASP *Dependency-Check*. O objetivo é impedir que atualizações automáticas ou pacotes comprometidos introduzam código malicioso e vulnerabilidades legadas no pipeline de CI/CD, fortalecendo a rastreabilidade e a resposta a incidentes.

5.5. Desafios Abertos

Apesar dos avanços, permanecem desafios técnicos relevantes. O predomínio de APIs (REST e GraphQL), SPAs e processamento *DOM-based* altera a superfície de ataque e dificulta a mitigação de falhas no cliente, como *DOM-based* XSS, exigindo sanitização contextual mais robusta no navegador. Em paralelo, APIs ocultas (*shadow APIs*) criam pontos não mapeados de risco e demandam descoberta contínua, políticas granulares de autorização e controles de taxa, uma vez que proteções estritamente no cliente podem ser contornadas por ataques diretos à API.

6. Conclusão

REVISAR TODO MUNDO JUNTO

Este trabalho demonstrou que a segurança em aplicações web consolidou-se como um pilar estratégico, transcendendo a camada perimetral de rede [Cloud Security Alliance 2024, Alwan and Younis 2021]. A defesa em profundidade surge como o princípio unificador fundamental, integrando a análise de código (SAST/IAST), testes dinâmicos (DAST) e proteção em *runtime* (RASP) para mitigar falhas desde o *design* até a execução [De Jesús Domínguez-García et al. 2023, Alsaedi 2023, Barth et al. 2008, Pahl and Jamshidi 2016].

Apesar dos avanços, identificou-se uma lacuna na literatura e na prática atual quanto à segurança de arquiteturas descentralizadas, como *shadow* APIs e o processamento complexo no lado do cliente (*DOM-based*), que dificultam o rastreamento automatizado de vulnerabilidades [Escape 2023, Tanveer 2025, Sureda Riera et al. 2025]. Além disso, embora a Inteligência Artificial (IA) seja promissora na detecção de anomalias, sua falta de interpretabilidade ainda limita a auditoria crítica em sistemas de larga escala [Yadav 2025a, Alsaedi 2023].

Como direções futuras, aponta-se a necessidade de amadurecer modelos de IA explicável aplicados à cibersegurança e a consolidação de arquiteturas *Zero Trust* baseadas em identidades dinâmicas [Yadav 2025a, of Standards and (NIST) 2020, Yadav 2025b]. A evolução para uma cultura *DevSecOps* plenamente automatizada é essencial para garantir que a agilidade no desenvolvimento não comprometa a integridade dos dados e a resiliência das infraestruturas modernas [Edmundson and Hartman 2022, De Jesús Domínguez-García et al. 2023].

7. Referencias TODO MUNDO TEM QUE REVISAR

Referências

- Alazmi, S. and De Leon, D. C. (2022). A systematic literature review on the characteristics and effectiveness of web application vulnerability scanners. *IEEE Access*, 10:33200–33219.
- Alsaedi, M. (2023). Deep learning technique-enabled web application firewall for the detection of web attacks. *Sensors*, 23(4):2073.
- Alwan, Z. S. and Younis, M. F. (2021). *An OWASP top ten driven survey on web application protection methods*, page 203–222. Springer.
- Appelt, D. (2014). Automated testing for sql injection vulnerabilities: an input mutation approach. In *International Symposium on Software Testing and Analysis (ISSTA)*, page 259–269, San Jose. ACM.
- Barth, A., Jackson, C., and Mitchell, J. C. (2008). Robust defenses for cross-site request forgery. In *ACM Conference on Computer and Communications Security (CCS)*, page 75–88, Alexandria, VA. ACM.
- Blessing, J., Specter, M. A., and Weitzner, D. J. (2021). You really shouldn't roll your own crypto: An empirical study of vulnerabilities in cryptographic libraries.
- Cloud Security Alliance (2024). Top threats to cloud computing. Technical report, Cloud Security Alliance (CSA). Acessado em: 16 mar. 2026.

- Cotroneo, D. (2025). Comparing effectiveness and efficiency of interactive application security testing (iast) and runtime application self-protection (rasp) tools in a large java-based system. *Empirical Software Engineering*.
- De Jesús Domínguez-García, A., Limón, X., Ocharán-Hernández, J. O., and Pérez-Arriaga, J. C. (2023). Security testing for web applications: A systematic literature review. In *2023 11th International Conference in Software Engineering Research and Innovation (CONISOFT)*, page 82–91.
- Doupé, A., Cova, M., and Vigna, G. (2010). *Detection of Intrusions and Malware, and Vulnerability Assessment*, volume 6201, page 111–131. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Edmundson, C. and Hartman, K. G. (2022). *SANS 2022 DevSecOps survey: creating a culture to significantly improve your organization's security posture*.
- Escape (2023). The state of graphql security 2023.
- Gupta, S. and Gupta, B. B. (2015). Cross-site scripting (xss) attacks and defense mechanisms: classification and state-of-the-art. *International Journal of System Assurance Engineering and Management*, 8(S1):512–530.
- Halfond, W. G. J., Viegas, J., and Orso, A. (2006). A classification of sql-injection attacks and countermeasures. In *IEEE International Symposium on Secure Software Engineering (ISSSE)*. IEEE.
- Heiderich, M., Schwenk, J., Frosch, T., Magazinius, J., and Yang, E. Z. (2013). mxss attacks: Attacking well-secured web-applications by using innerhtml mutations. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security - CCS '13*, page 777–788, Berlin, Germany. ACM Press.
- Kusnana, K., Gymnastiar, P., and Riswanto, B. (2025). Analisis kerentanan insecure direct object reference (idor) dan broken access control pada aplikasi invoice berbasis web. *Digital Transformation Technology*, 5(2):106–114.
- Lin, X., Zavarisky, P., Ruhl, R., and Lindskog, D. (2009). Threat modeling for csrf attacks. In *2009 International Conference on Computational Science and Engineering*, volume 3, pages 486–491.
- Mello, E. R., Wangham, M. S., da Silva Fraga, J., and Camargo, E. (2006). *Segurança em Serviços Web*.
- of Standards, N. I. and (NIST), T. (2020). *NIST special publication 800-207: zero trust architecture*. Gaithersburg.
- OWASP Foundation (2021). Owasp top 10:2021.
- Pahl, C. and Jamshidi, P. (2016). Microservices: a systematic mapping study. In *International Conference on Cloud Computing and Services Science (CLOSER)*, page 137–146, Rome. SCITEPRESS.
- Sacramento, L., Ítalo Cunha, Cardoso, G., Souza, A., Franco, A., and Oliveira, L. (2024). Automação de autenticação para testes de segurança em aplicações web no zap. In *Anais do XXIV Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg 2024)*, page 760–766, Porto Alegre, RS, Brasil. Sociedade Brasileira de Computação - SBC.

- Sarmah, U., Bhattacharyya, D., and Kalita, J. (2018). A survey of detection methods for xss attacks. *Journal of Network and Computer Applications*, 118:113–143.
- Shahriar, H. and Zulkernine, M. (2011). Taxonomy and classification of automatic monitoring of program security vulnerability exploitations. *Journal of Systems and Software*, 84(2):379–393.
- Shar, L. K. and Tan, H. B. K. (2016). Securing web applications from injection and logic vulnerabilities: approaches and challenges. *Information and Software Technology*, 74:160–180.
- Sureda Riera, T., Ramón Bermejo Higuera, J., Bermejo Higuera, J., Antonio Sicilia Montalvo, J., and Martínez Herráiz, J. J. (2025). Twenty-two years since revealing cross-site scripting attacks: a systematic mapping and a comprehensive survey. *Computer Modeling in Engineering & Sciences*, 133(3):579–599.
- Tanveer, H. (2025). Towards secure apis: a survey on restful api vulnerability detection. *Computers, Materials & Continua*, 84(3).
- Yadav, D. (2025a). Cutting-edge approaches in intrusion detection systems: deep learning, reinforcement learning, and ensemble techniques. *Iranian Journal of Science and Technology, Transactions of Electrical Engineering*.
- Yadav, D. (2025b). Zero trust security architecture in microservices via mtls + oauth 2.0. *International Journal of Advanced Research in Computer Science and Electronics Engineering*.
- Yadav, D., Gupta, D., Singh, D., Kumar, D., and Sharma, U. (2018). Vulnerabilities and security of web applications. In *2018 4th International Conference on Computing Communication and Automation (ICCCA)*, page 1–5.